Recursive Convergent Inference (RCI):
Bottom-Up Module Expansion via Output Convergence

Alex Anokhin
Independent Researcher
olanokhin@gmail.com

Code & Colab: github.com/olanokhin/rci-inference

March 2026

Abstract

We propose **Recursive Convergent Inference (RCI)**, an architectural principle for neural network inference in which the set of active computational modules expands monotonically from a minimal seed subset until empirical convergence of the model's next-token output distribution. Unlike existing adaptive computation methods that determine *when to halt* a fixed computation, RCI determines *when additional computation is warranted*—progressively incorporating additional modules based on inter-module affinities until output stability is reached. Inference terminates when a hybrid distributional stability metric falls below a threshold, with additional safeguards against premature convergence. Crucially, RCI requires no learned halting signal, external verifier, or task-complexity pre-estimator; stopping emerges directly from the internal stability of the model's own output distribution. As each inference process is self-contained, RCI naturally enables parallel multi-task execution over shared read-only weights. This offers a complementary perspective to existing test-time compute scaling by shifting from *external scaling* (longer outputs, multiple samples, verifier-guided search) to *internal scaling* over the active parameter subgraph.

We instantiate RCI on OLMoE-1B-7B-0924—a fully open model with 64 discrete experts. Empirical evaluation on reasoning benchmarks of varying difficulty (GSM8K, MATH, MMLU hard subsets, n=150) demonstrates that all pairwise difficulty comparisons are statistically significant: hard tasks accumulate significantly greater total KL divergence than easy (Mann-Whitney U=1677, p=0.002) and medium tasks (U=2106, p<0.001), with a statistically significant monotonic correlation (Spearman ρ=0.22, p=0.007). Notably, RCI's complexity metric diverges from human-defined difficulty labels, suggesting it captures *model-relative computational demand* rather than task difficulty in the abstract. We release a fully reproducible implementation as a Google Colab notebook alongside this paper.

Keywords: adaptive inference, mixture-of-experts, output convergence, dynamic subgraph expansion, test-time compute, efficient inference

1 Introduction

The computational cost of inference in large neural networks is conventionally fixed by architecture: a model with L layers and N parameters applies largely the same amount and structure of computation to every input, regardless of whether the task requires synthesizing multi-step reasoning chains or answering a simple factual query. This rigidity constitutes a fundamental inefficiency—a direct retrieval question consumes the same resources as a complex mathematical proof.

Recent advances have pursued two complementary directions. *External test-time scaling* allocates additional compute by extending token sequences or sampling multiple trajectories, selecting among them via process reward models or majority voting [Snell et al., 2024, Muennighoff et al., 2025]. *Adaptive computation* methods reduce wasted compute by learning early exits or layer skips from a fixed graph, as in Adaptive Computation Time (ACT) [Graves, 2016], Universal Transformers [Dehghani et al., 2019], PonderNet [Banino et al., 2021], and Mixture-of-Depths [Raposo et al., 2024]. Both approaches operate **top-down**: they begin with a maximal computational graph and learn when to stop or what to remove. The symmetric direction—beginning from a minimal active parameter subset and expanding it bottom-up as a function of task complexity—has remained comparatively underexplored.

We propose **Recursive Convergent Inference (RCI)**, an inference principle that addresses this gap. Rather than fixing the active parameter set and learning when to halt, RCI begins from a minimal seed subset of modules and

expands it monotonically ($M_n \subseteq M_{n+1}$) until the model's next-token output distribution converges. The stopping criterion is intrinsic: additional modules no longer meaningfully alter the output distribution, without relying on an explicit learned halting signal or external evaluator.

This formulation induces several properties. First, computation is *task-adaptive by construction*: simple inputs converge early over small subgraphs, while complex reasoning recruits progressively larger portions of the network. Second, each inference process is *self-contained*, operating over shared read-only weights and potentially enabling parallel multi-task execution. Third, it is *estimator-free*: termination arises from the distributional stability of the model's own output. More broadly, this suggests that computation in large networks can be allocated dynamically at the level of parameter activation, rather than being fixed by architecture or governed by external sampling strategies.

Contributions. We make the following contributions:

1. **Recursive Convergent Inference (RCI)**—a novel inference principle formulating computation as monotonic module-set expansion ($M_n \subseteq M_{n+1}$) until empirical output convergence, providing a bottom-up counterpart to existing top-down adaptive methods.

2. **Module affinity graph**—a lightweight static graph derived from expert co-activation patterns in MoE models, used to guide structured expansion of the active module set.

3. **Convergence stopping criterion**—a hybrid output stability metric combining KL divergence, cosine similarity, and output entropy over a rolling window, conditioned on a Confidence Margin to mitigate false convergence without external supervision.

4. **Empirical validation**—an open, reproducible implementation on OLMoE-1B-7B-0924 demonstrating that the computational effort required for convergence (measured via cumulative KL divergence across expansion steps) scales with task complexity. All pairwise difficulty comparisons are statistically significant across $n{=}150$ examples (50 per tier—GSM8K, MATH algebra, MMLU hard subsets): hard tasks accumulate significantly greater cumulative KL divergence than easy tasks (Mann-Whitney $U{=}1677$, $p{=}0.002$) and medium tasks ($U{=}2106$, $p{<}0.001$), with a statistically significant monotonic correlation (Spearman $\rho{=}0.22$, $p{=}0.007$, $n{=}150$). Notably, RCI's complexity metric diverges from human-defined difficulty labels, indicating model-relative computational demand rather than task difficulty in the abstract.

2 Related Work

RCI lies at the intersection of five active research directions: test-time compute scaling, adaptive computation, sparse mixture-of-experts routing, fixed-point iterative inference, and graph-based computation.

Test-time compute scaling. Scaling computation at inference time has emerged as a primary axis for improving reasoning in large language models [Snell et al., 2024, Agarwal et al., 2025]. Existing approaches fall into two categories. *Serial scaling* increases compute by extending token sequences (Chain-of-Thought, iterative revision). *Parallel scaling* samples multiple output trajectories and selects among them via verifiers or reward models (Best-of-N, Tree Search, MCTS-based planning). Self-consistency [Wang et al., 2023] samples multiple independent outputs and selects by majority vote. Simple test-time scaling [Muennighoff et al., 2025] demonstrates that SFT on a small set of reasoning traces combined with budget forcing achieves strong scaling behaviour. RCI differs by scaling computation *internally*—expanding the active parameter subgraph per forward pass without altering context length or requiring any external verifier.

Adaptive computation and halting. ACT [Graves, 2016] introduces a learned scalar halting probability for recurrent networks. The Universal Transformer [Dehghani et al., 2019] extends this to transformers via weight-shared recurrent layers. PonderNet [Banino et al., 2021] reformulates halting probabilistically. Mixture-of-Depths [Raposo et al., 2024] learns per-token binary skip decisions. More recent work includes AdaPonderLM [Song et al., 2025] and ADEPT [Yoo et al., 2026]. Answer Convergence [Liu and Wang, 2025] detects when outputs stabilise across reasoning steps—the closest prior use of a convergence signal for stopping. All of the above focus on *halting*; RCI asks the symmetric question of *when additional computation is warranted*.

Mixture-of-Experts. The Switch Transformer [Fedus et al., 2022] established sparse MoE as a practical scaling paradigm. OLMoE [Muennighoff et al., 2024] demonstrated that open MoE models with 64 experts and top-8 routing match larger dense counterparts. OpenMoE [Xue et al., 2024] confirmed strong expert specialisation during pre-training. Standard and dynamic MoE methods make a single routing decision per forward pass. RCI treats the active expert set as a dynamically expanding frontier across recursive steps, repurposing co-activation statistics as a static affinity graph—a structural use of MoE routing information not present in prior work.

Fixed-point and iterative inference. Deep Equilibrium Models [Bai et al., 2019] formulate inference as finding a fixed point of a weight-tied layer. Looped Transformers and recurrent-depth architectures demonstrate that iterating a single parameterised block can scale test-time computation. SELF-Transformer [Mathur et al., 2025] applies fixed-point iteration within a single self-attention layer. Inner Thinking Transformer [Chen et al., 2025] interprets depth as internal reasoning steps. All existing fixed-point methods refine representations within a *fixed* computational structure; RCI differs in that convergence is achieved through *expansion* of the active module set.

Hardware-efficient sparse operations. Flash-KMeans [svg-project, 2025] demonstrates that IO-aware kernel engineering can yield 30–200× speedups for sparse operations previously considered hardware-unfriendly, suggesting a concrete engineering path toward reducing RCI's wall-clock overhead without algorithmic changes.

Summary of distinctions. Table 1 summarises the key distinctions.

Table 1: Comparison of RCI with related methods.

Method	Dyn. Depth	Monotonic Width	Conv. Stop	No Pre-est.	Parallel
ACT / UT / PonderNet	✓	×	×	×	×
Mixture-of-Depths	✓	×	×	×	×
Answer Convergence	✓	×	✓	×	×
Switch / OLMoE	×	×	×	×	×
SELF-Transformer	✓	×	✓	✓	×
Best-of-N / MCTS	×	×	ext. budget	×	×
RCI (ours)	✓	✓	✓	✓	✓

3 Recursive Convergent Inference

3.1 Core Principle

RCI formulates inference as monotonic module-set expansion until empirical output convergence. In contrast to existing top-down adaptive methods, RCI operates bottom-up: computation begins at a minimal seed subset and grows as warranted by the task.

3.2 Seed Initialisation

Seeds define the initial subset of active computational modules. In the PoC, seeds are initialised as the top-8 most frequently activated experts on the first standard forward pass. In the full architecture, a lightweight input classifier (embedding + linear router) maps task type to domain-specific module clusters, trained with an auxiliary loss including $\lambda_4 \cdot \mathrm{Var}(M_{\mathrm{active}})$ to encourage distinct subgraph activation across task types.

3.3 Recursive Expansion

The affinity graph encodes co-activation frequency between modules:

$$\mathrm{affinity}[i][j] \mathrel{+}= 1 \quad \text{if experts } i, j \text{ are co-activated within the same sequence.}$$

The expansion rule is:

$$M_0 = \text{seeds} \tag{1}$$

$$M_{n+1} = M_n \cup \text{top-k}(\text{neighbors}(M_n),\ \text{weighted by affinity}), \quad M_n \subseteq M_{n+1} \tag{2}$$

Implementation proceeds via a gating mask:

```
gating_logits[experts_not_in_M] = -inf
current_k = min(base_k * (2 ** step), total_experts)
```

3.4 Convergence Criterion

$$\Delta_n = \alpha \cdot \mathrm{KL}(P_n \,\|\, P_{n-1}) + \beta \cdot \cos(h_n, h_{n-1}) + \gamma \cdot H(P_n) \tag{3}$$

$$\text{Convergence: } \frac{1}{W} \sum_{i=n-W+1}^{n} \Delta_i < \varepsilon \ \text{ and } \ \big[P(\text{top-1}) - P(\text{top-2})\big] \geq \theta \tag{4}$$

The Confidence Margin guards against false convergence at local plateaus without requiring an external estimator.

3.5 Training Objective

$$\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda_1 \mathcal{L}_{\text{conv}} + \lambda_2 \mathcal{L}_{\text{sparse}} + \lambda_3\, \mathrm{MI}(\text{seeds},\ M_{\text{final}}) + \lambda_4\, \mathrm{Var}(M_{\text{active}}) \tag{5}$$

4 Experiments

4.1 Setup

Model. OLMoE-1B-7B-0924 [Muennighoff et al., 2024]: 64 experts per MoE layer, top-8 routing, 1B active / 7B total parameters, Apache 2.0 licence. All experiments on a single NVIDIA T4 GPU (16 GB) via Google Colab.

Affinity graph. 500 calibration examples from C4 [Raffel et al., 2020]. Co-activation counts are L_∞-normalised to $[0, 1]$. Construction requires approximately 10 minutes on T4.

RCI loop. Seeds: top-8 most activated experts on first pass. Expansion: BFS over affinity graph, k=4 neighbours per step. Gating: dynamic top-k doubling (8→16→32→64). Convergence: ε=0.093 (calibrated as the 20th percentile of KL values over 5 calibration tasks), W=3, max steps = 15.

Benchmarks.

- **Easy**: GSM8K [Cobbe et al., 2021] — grade school mathematics word problems
- **Medium**: MATH algebra [Hendrycks et al., 2021a]
- **Hard**: MMLU hard subsets [Hendrycks et al., 2021b] — college physics, chemistry, abstract algebra, formal logic

50 examples per tier, n=150 total.

4.2 Results

All pairwise Mann-Whitney U tests are statistically significant: hard vs. easy (U=1677, p=0.002), hard vs. medium (U=2106, p<0.001), easy vs. medium (U=1788, p<0.001). The overall Spearman rank correlation is ρ=0.22 (p=0.007, n=150).

Table 2: Cumulative KL divergence (AUC) by difficulty tier (n=150, 50 per tier).

Difficulty	Benchmark	n	Mean AUC	Std
Easy	GSM8K	50	10.728	2.808
Medium	MATH (algebra)	50	8.956	1.688
Hard	MMLU hard	50	11.987	2.537

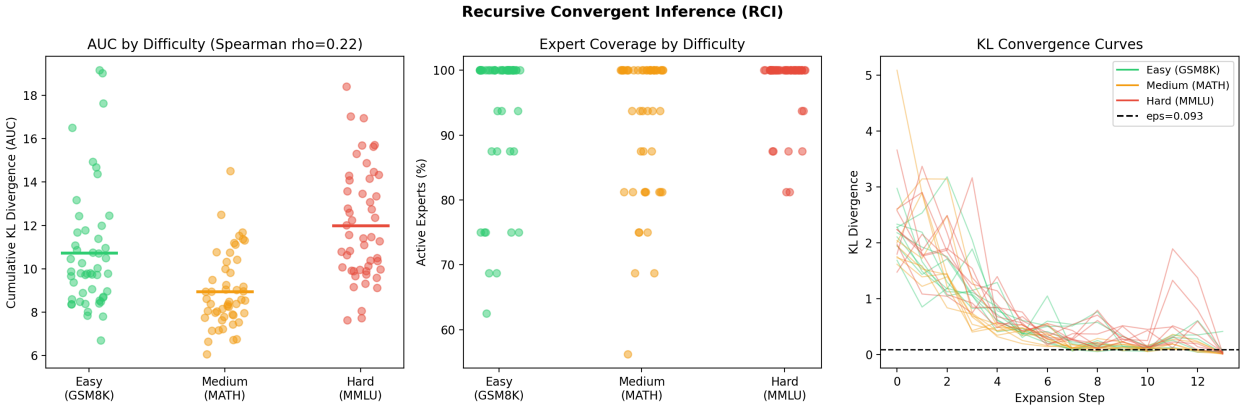

Figure 1: **Left:** Cumulative KL divergence (AUC) per difficulty tier; horizontal bars indicate group means. **Centre:** Active expert coverage (%) per task. **Right:** Representative KL convergence curves; dashed line indicates ε=0.093.

4.3 Model-Relative Complexity

A notable finding is that MATH algebra problems (human label: medium) require less cumulative KL divergence than GSM8K word problems (human label: easy) for this model (8.96 vs. 10.73, p<0.001). We interpret this as evidence that RCI measures **model-relative computational demand**—the amount of information a specific model must extract from its own weight structure to stabilise its output distribution on a specific input—rather than task difficulty in the abstract. This dissociation between model-internal and human-perceived difficulty warrants further investigation across model families and scales.

4.4 Ablation

Affinity-weighted BFS and random BFS yield equivalent average steps on this pre-trained model without RCI-specific fine-tuning. This is consistent with the hypothesis that the full benefit of structured expansion requires training under the objective in Equation 5.

5 Limitations and Future Work

Wall-clock latency. Each RCI step is a full forward pass; real-world speedup requires fused sparse kernels. Recent IO-aware GPU primitives [svg-project, 2025] suggest this overhead may be substantially reducible without algorithmic changes.

Scale. The PoC uses 1B active parameters. Behaviour at 8B–70B scale remains unexamined.

Static affinity graph. Computed offline over 500 calibration examples; may be suboptimal under distribution shift. Dynamic affinity estimation is a natural extension.

Parallel heads. Multi-head parallelism requires multi-device infrastructure and is not evaluated in the present PoC.

Convergence guarantees. RCI relies on empirical convergence. Formal guarantees via contraction mapping are not pursued.

Future directions. (i) Attention-based pooling to mitigate oversmoothing at high module coverage; (ii) training from scratch under the full RCI objective; (iii) validation across MoE families (Mixtral, Qwen-MoE, DeepSeek-MoE); (iv) extension to multi-modal architectures.

6 Conclusion

We introduced Recursive Convergent Inference (RCI), an inference principle in which the set of active computational modules expands monotonically from a minimal seed subset until empirical convergence of the model's next-token output distribution. In contrast to existing adaptive computation methods that operate top-down—starting from a maximal graph and learning when to halt—RCI operates bottom-up, growing computation as warranted by the task rather than pruning it post-hoc. Stopping requires no learned halting signal, external verifier, or task-complexity pre-estimator; it emerges directly from the distributional stability of the model's own output.

We instantiated RCI on OLMoE-1B-7B-0924, a fully open mixture-of-experts model with 64 discrete experts, and provided a reproducible implementation via Google Colab. Empirical evaluation across reasoning benchmarks (GSM8K, MATH algebra, MMLU hard subsets, n=150, 50 per tier) reveals two findings. First, all pairwise difficulty comparisons are statistically significant: hard tasks accumulate significantly greater cumulative KL divergence than easy (U=1677, p=0.002) and medium tasks (U=2106, p<0.001), with a statistically significant monotonic correlation overall (Spearman ρ=0.22, p=0.007, n=150). Second, and notably, RCI's complexity metric diverges from human-defined difficulty labels: MATH algebra problems require less cumulative KL divergence than GSM8K word problems for this model (8.96 vs. 10.73, p<0.001), suggesting an intriguing dissociation between model-internal and human-perceived difficulty that warrants further study across architectures and scales.

We believe RCI offers a complementary perspective to the growing literature on test-time compute scaling: that inference efficiency can be achieved not only by generating fewer tokens or selecting among multiple samples, but by dynamically allocating computation at the level of parameter activation itself.

References

Rishabh Agarwal et al. The art of scaling test-time compute. *arXiv preprint arXiv:2512.02008*, 2025.

Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. Deep equilibrium models. *Advances in Neural Information Processing Systems*, 2019.

Andrea Banino, Jan Balaguer, and Charles Blundell. Pondernet: Learning to ponder. *arXiv preprint arXiv:2107.05407*, 2021.

Yilong Chen et al. Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking. In *Annual Meeting of the Association for Computational Linguistics*, 2025.

Karl Cobbe et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(1):1–39, 2022.

Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.

Dan Hendrycks et al. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021a.

Dan Hendrycks et al. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2021b.

Wei Liu and Fang Wang. Answer convergence as a signal for early stopping in reasoning. In *Empirical Methods in Natural Language Processing*, 2025.

Priya Mathur et al. SELF-transformer: Iterative fixed-point self-attention. *arXiv preprint arXiv:2507.13569*, 2025.

Niklas Muennighoff, Luca Soldaini, Dirk Groeneveld, Kyle Lo, Jacob Morrison, Sewon Min, Weijia Shi, Pete Walsh, Oyvind Tafjord, Nathan Lambert, et al. OLMoE: Open mixture-of-experts language models. *arXiv preprint arXiv:2409.02060*, 2024.

Niklas Muennighoff et al. s1: Simple test-time scaling. In *Empirical Methods in Natural Language Processing*, 2025.

Colin Raffel et al. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. In *International Conference on Machine Learning*, 2024.

Charlie Snell et al. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

Zifei Song et al. AdaPonderLM: Gated pondering language models with token-wise adaptive depth. *arXiv preprint arXiv:2603.01914*, 2025.

svg-project. Flash-KMeans: IO-aware GPU implementation of exact k-means. *GitHub*, 2025. URL https://github.com/svg-project/flash-kmeans.

Xuezhi Wang et al. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.

Fuzhao Xue et al. OpenMoE: An early effort on open mixture-of-experts language models. *arXiv preprint arXiv:2402.01739*, 2024.

Jaehun Yoo et al. ADEPT: Adaptive dynamic early-exit process for transformers. *arXiv preprint arXiv:2601.03700*, 2026.